

Project Description

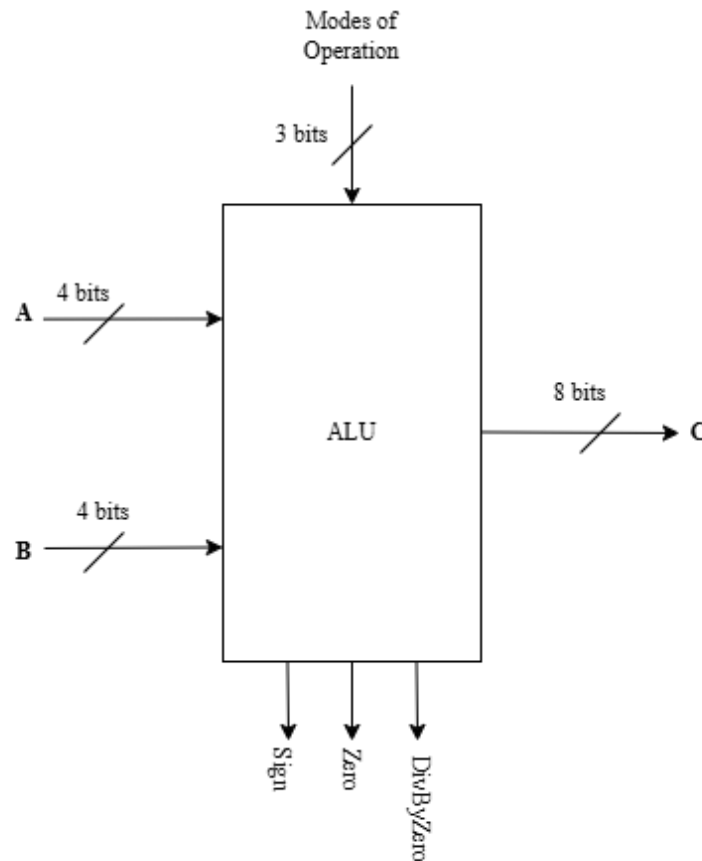
Project Overview

In this project, each team will design, simulate, and verify a 5-operation Arithmetic Logic Unit (ALU) using two design flows:

- A. LTSpice CMOS schematic design
- B. Verilog structural design with ModelSim simulation

The ALU must support these five operations for signed numbers:

- 1. Addition
- 2. Subtraction
- 3. Multiplication
- 4. Modulus (remainder)
- 5. Arithmetic Shifts Right



Operations Details

The Arithmetic Unit operates on two 4-bit signed inputs, A and B, encoded in two's complement format for negative numbers, along with a control input called **Mode of Operation**, which determines the arithmetic function performed on A and B:

- Addition: $C = A + B$
 - Both A and B are 4-bit signed numbers.
 - The output Result is an 8-bit signed number.
 - Sign extension is applied to correctly represent the sign in the larger bit-width result.
- Subtraction: $C = A - B$
 - Both A and B are 4-bit signed numbers.
 - The output Result is an 8-bit signed number.
 - Sign extension and two's complement subtraction are used to handle negative differences.
- Multiplication: $C = A \times B$
 - Both A and B are 4-bit signed numbers.
 - The output Result is an 8-bit signed number.
 - The product fits within 8 bits after multiplication of two 4-bit signed numbers.
- Modulus (remainder): $C = A \% B$
 - A, B, and Result are 4-bit signed numbers, but Result is presented as an 8-bit signed number.
 - Division by zero ($B = 0$) is forbidden; Result = 0 and the "Divide by Zero" flag is asserted.
 - The sign of the remainder matches that of A.
 - Examples:
 - $+2 \% +3 = +2$
 - $+2 \% -3 = +2$
 - $-2 \% +3 = -2$
 - $-2 \% -3 = -2$
- Arithmetic Shift Right (ASR): $C = A >>> B$
 - Input A is a 4-bit signed number
 - Input B is a 4-bit unsigned number indicating the amount of shift.
 - The result is **A** shifted right by **B** bits with sign extension, preserving the sign bit.
 - The output Result is an 8-bit signed number with proper sign extension.

All negative numbers use two's complement representation, where the most significant bit indicates the sign. The outputs are sign-extended to 8 bits to correctly reflect the signed results.

Flags

- **Sign Flag:** This flag indicates if the result is negative. It is set to 1 when the most significant bit (MSB) of the 8-bit signed result is 1, which signifies a negative number in two's complement representation. Otherwise, it is set to 0.
- **Zero Flag:** This flag indicates if the result is zero. It is set to 1 when all bits of the 8-bit signed result are zero. Otherwise, it is set to 0.
- **Divide by Zero Flag:** This flag signals an invalid division operation. It is set to 1 whenever the divisor operand B is zero during a division or remainder operation. In such a case, the result is forced to 0. Otherwise, this flag remains 0.

These flags provide essential status signals following arithmetic operations, helping to interpret the signed results properly while signaling exceptional conditions like division by zero.

Project Requirements

A. CMOS & LTSpice

1. Implement each operation at transistor level using CMOS logic (Hint: Look up bus notation in LTSpice, it may be useful).
2. Simulate using LTSpice:
 - Transient analysis
 - Propagation delay measurement (Worst-Case and Best-Case delay for each operation and for the whole ALU)
 - Power dissipation (for each operation and for the whole ALU)
3. Generate clean schematics and simulation waveforms.
4. Use correct CMOS logic design rules (no floating nodes, proper sizing for symmetric delay, and well connections).

B. Verilog & ModelSim

1. Write **structural** Verilog (gates only) for the entire ALU.
2. Create a well-structured testbench.
3. Simulate using ModelSim:
 - Demonstrate each implemented operation
 - Verify functionally correct outputs
4. Operators like (+, -, *, %, >>, >>> and {}) are strictly **forbidden**.

Deliverables

A report (PDF) for part A:

1. Team workload distribution (use the example in the appendix as a guide)
2. CMOS design for each operation (schematics & explanations)
3. LTSpice waveforms & analysis (Delay & Power)

A report (PDF) for part B:

1. Team workload distribution (use the example in the appendix as a guide)
2. Verilog architecture & module hierarchy
3. ModelSim waveforms

A presentation to for the project discussion.

Rules & Notes

1. Team size is 5, work must be distributed fairly.
2. Plagiarism between teams results in zero marks.
3. Creativity is encouraged: teams may optimize for performance, area, or power.

Appendix

Team Workload Distribution: Part A (CMOS & LTSpice)

Team Member	Assigned Tasks
Member 1 Name	CMOS schematic & LTSpice simulation for Addition Extracting propagation delays and power consumption for the entire ALU
Member 2 Name	CMOS schematic & LTSpice simulation for Subtraction Preparing flags Logic
Member 3 Name	CMOS schematic & LTSpice simulation for Multiplication
Member 4 Name	CMOS schematic & LTSpice simulation for Modulus (remainder)
Member 5 Name	CMOS schematic & LTSpice simulation for Arithmetic Shift Right (ASR)

Shared Responsibilities

1. Integrating all CMOS blocks into the full ALU in LTSpice
2. Preparing the ALU Testbench.
3. Contributing to Part A report and presentation

Team Workload Distribution: Part B (Verilog & ModelSim)

Team Member	Assigned Tasks
Member 1 Name	Verilog structural module & ModelSim tests for Addition Developing the main ALU testbench
Member 2 Name	Verilog structural module & ModelSim tests for Subtraction Preparing the flags logic (Sign, Zero, Divide-by-Zero)
Member 3 Name	Verilog structural module & ModelSim tests for Multiplication
Member 4 Name	Verilog structural module & ModelSim tests for Modulus (remainder)
Member 5 Name	Verilog structural module & ModelSim tests for ASR

Shared Responsibilities

1. Integrating all Verilog modules into the complete ALU
2. Contributing to Part B report and presentation